

Week 4 — Assignment Problems (Arrays, Category C) — Solutions

Category C · 5 assignment problems · arrays, prefix sums, two pointers and binary search

Name: Gursimran Bajwa | **Registration No:** RA2511003011289

Language: Java | **Problems solved:** 5 / 5

Problem 1 — Product of Array Except Self

Problem Statement

A pricing engine needs, for every product in a bundle, the combined price of every other product — computed for all products at once, without ever dividing by the current product's own price, since a price could legitimately be zero.

Approach

- The answer at index i is (product of everything left of i) $\times$ (product of everything right of i). Two passes build exactly those two halves.
- Forward pass: a running leftProduct is written into answer[i] before being multiplied by nums[i], so answer[i] holds the product of everything strictly to the left. Index 0 correctly gets 1.
- Backward pass: a running rightProduct multiplies into each answer[i] the same way, combining both halves in place.
- Zeros need no special handling at all — a single zero simply makes every other position 0 while the zero's own position keeps the product of the rest, which is why [-1, 1, 0, -3, 3] gives [0, 0, 9, 0, 0].
- $O(n)$ time with $O(1)$ extra space beyond the output array, and no division anywhere.

Java Program — ProductOfArrayExceptSelf.java

```
/**
 * A1 - Product of Array Except Self (Bundle Pricing Engine)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 * O(n) time, no division anywhere, O(1) extra space beyond the output array.
 */
public class ProductOfArrayExceptSelf {

    /* Forward pass stores the product of everything to the left of i;
       backward pass multiplies in the product of everything to the right. */
    public static int[] productExceptSelf(int[] nums) {
        int[] answer = new int[nums.length];

        int leftProduct = 1;
        for (int i = 0; i < nums.length; i++) {
            answer[i] = leftProduct;           // nothing to the left of index 0
            leftProduct = leftProduct * nums[i];
        }

        int rightProduct = 1;
        for (int i = nums.length - 1; i >= 0; i--) {
            answer[i] = answer[i] * rightProduct; // combine left and right halves
            rightProduct = rightProduct * nums[i];
        }

        return answer;
    }

    private static String format(int[] array) {
        StringBuilder text = new StringBuilder("[");
        for (int i = 0; i < array.length; i++) {
            text.append(array[i]);
            if (i < array.length - 1) text.append(", ");
        }
        text.append("]");
        return text.toString();
    }
}
```

```
    }  
    return text.append("]").toString();  
}  
  
public static void main(String[] args) {  
    int[][] inputs = { { 1, 2, 3, 4 }, { -1, 1, 0, -3, 3 }, { 2, 3 } };  
  
    for (int[] nums : inputs) {  
        System.out.println("nums = " + format(nums)  
            + " -> " + format(productExceptSelf(nums)));  
    }  
}
```

Output

```
nums = [1, 2, 3, 4] -> [24, 12, 8, 6]  
nums = [-1, 1, 0, -3, 3] -> [0, 0, 9, 0, 0]  
nums = [2, 3] -> [3, 2]
```

Problem 2 — Maximum Subarray (Kadane's Algorithm)

Problem Statement

A trader has a year of daily profit-or-loss figures, some positive and some negative, and wants the single best contiguous stretch of days — the run of consecutive days whose combined total is the highest possible.

Approach

- Kadane's algorithm makes one decision per element: extend the current running subarray, or abandon it and restart from the current element — whichever gives the larger sum.
- Extending only pays off when $\text{currentSum} + \text{nums}[i]$ beats $\text{nums}[i]$ on its own, which happens exactly when currentSum is positive. A negative running sum is always worth discarding.
- Both currentSum and bestSum are seeded with $\text{nums}[0]$ rather than 0. This is the detail that makes an all-negative array return its largest single element (-1 for $[-3, -1, -2]$) instead of an incorrect 0.
- $O(n)$ time and $O(1)$ space. The $O(n \log n)$ divide-and-conquer alternative splits the array and combines the best left, best right and best crossing subarray — correct, but strictly slower.

Java Program — MaximumSubarray.java

```
/**
 * A2 - Maximum Subarray (Trader's Best Stretch of Days)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 * Kadane's algorithm -  $O(n)$  time,  $O(1)$  extra space.
 */
public class MaximumSubarray {

    /* At each element: extend the running subarray, or restart from this element. */
    public static int maxSubArray(int[] nums) {
        int currentSum = nums[0];
        int bestSum = nums[0]; // seeded with a real element, so all-negative works

        for (int i = 1; i < nums.length; i++) {
            if (currentSum + nums[i] > nums[i]) {
                currentSum = currentSum + nums[i]; // extending still pays off
            } else {
                currentSum = nums[i]; // abandon the past, start fresh here
            }

            if (currentSum > bestSum) {
                bestSum = currentSum;
            }
        }
        return bestSum;
    }

    private static String format(int[] array) {
        StringBuilder text = new StringBuilder("[");
        for (int i = 0; i < array.length; i++) {
            text.append(array[i]);
            if (i < array.length - 1) text.append(", ");
        }
        return text.append("]").toString();
    }

    public static void main(String[] args) {
        int[][] inputs = { { -2, 1, -3, 4, -1, 2, 1, -5, 4 }, { -3, -1, -2 }, { 5, 4, -1, 7, 8 } };

        for (int[] nums : inputs) {
            System.out.println("nums = " + format(nums)
                + " -> Max Subarray Sum: " + maxSubArray(nums));
        }
    }
}
```

Output

```
nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4] -> Max Subarray Sum: 6
nums = [-3, -1, -2] -> Max Subarray Sum: -1
nums = [5, 4, -1, 7, 8] -> Max Subarray Sum: 23
```

Problem 3 — 3Sum

Problem Statement

A budgeting tool must find every distinct combination of exactly three transactions that cancel out to zero, without ever reporting the same combination of amounts twice — even when it could be picked out of the list in more than one way.

Approach

- Sorting first is what makes the whole approach work: it puts duplicates next to each other so they can be skipped, and it lets two pointers navigate by comparison.
- For each index i , two pointers start just after i and at the end of the array, moving inward. If the triplet sums below zero the left pointer rises; above zero the right pointer falls; at exactly zero the triplet is recorded.
- Duplicates are skipped at all three levels — the outer loop skips a repeated first value, and after a hit both pointers are advanced past their own repeats. This is the single most common source of bugs in this problem.
- Because the array is sorted, each recorded triplet is already in ascending order, so $[-1, -1, 2]$ and $[-1, 0, 1]$ come out in a stable, comparable form.
- $O(n^2)$ time overall: an $O(n \log n)$ sort plus an $O(n)$ two-pointer scan for each of the n elements.

Java Program — ThreeSum.java

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * A3 - 3Sum (Budgeting Tool: Transactions That Cancel Out)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 * Sort + two pointers -  $O(n^2)$  time after an  $O(n \log n)$  sort.
 */
public class ThreeSum {

    /** For each element, two pointers sweep the rest of the sorted array for a matching pair. */
    public static int[][] threeSum(int[] nums) {
        Arrays.sort(nums); // sorting makes duplicates adjacent
        List<int[]> triplets = new ArrayList<>();

        for (int i = 0; i < nums.length - 2; i++) {

            // Skip a repeated first value, or the same triplet would be reported twice.
            if (i > 0 && nums[i] == nums[i - 1]) continue;

            int left = i + 1;
            int right = nums.length - 1;

            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];

                if (sum == 0) {
                    triplets.add(new int[] { nums[i], nums[left], nums[right] });

                    // Move both pointers past their duplicate values.
                    while (left < right && nums[left] == nums[left + 1]) left++;
                    while (left < right && nums[right] == nums[right - 1]) right--;
                    left++;
                    right--;
                } else if (sum < 0) {
                    left++; // need a larger sum
                } else {
                    right--; // need a smaller sum
                }
            }
        }

        return triplets.toArray(new int[triplets.size()][]);
    }

    private static String format(int[] array) {
        StringBuilder text = new StringBuilder("[");
    }
}
```

```
        for (int i = 0; i < array.length; i++) {
            text.append(array[i]);
            if (i < array.length - 1) text.append(", ");
        }
        return text.append("]").toString();
    }

    private static String format(int[][] rows) {
        StringBuilder text = new StringBuilder("[");
        for (int i = 0; i < rows.length; i++) {
            text.append(format(rows[i]));
            if (i < rows.length - 1) text.append(", ");
        }
        return text.append("]").toString();
    }

    public static void main(String[] args) {
        int[][] inputs = { { -1, 0, 1, 2, -1, -4 }, { 0, 0, 0 }, { -2, 0, 1, 1, 2 } };

        for (int[] nums : inputs) {
            String before = format(nums);
            System.out.println("nums = " + before + " -> " + format(threeSum(nums)));
        }
    }
}
```

Output

```
nums = [-1, 0, 1, 2, -1, -4] -> [[-1, -1, 2], [-1, 0, 1]]
nums = [0, 0, 0] -> [[0, 0, 0]]
nums = [-2, 0, 1, 1, 2] -> [[-2, 0, 2], [-2, 1, 1]]
```

Problem 4 — Subarray Sum Equals K

Problem Statement

A warden reviewing a semester's daily attendance-change log (some days net positive, some negative) wants to know how many different contiguous stretches of days had a net change of exactly k , across the whole log.

Approach

- The key identity is that the sum of the subarray ending at j and starting just after i equals $\text{prefixSum}[j] - \text{prefixSum}[i]$. So at each position the question becomes: how many earlier prefix sums equal $(\text{runningSum} - k)$?
- A `HashMap` of prefix-sum frequencies answers that in $O(1)$, so the whole scan is a single $O(n)$ pass.
- The map is seeded with `prefixCounts.put(0, 1)` — the empty prefix. Without this base case, subarrays that start at index 0 would never be counted.
- A sliding window cannot be used here. Window techniques rely on the running sum growing as the window widens, and negative values break that monotonicity, so shrinking the window is no longer guaranteed to reduce the sum.
- $O(n)$ time and $O(n)$ space.

Java Program — SubarraySumEqualsK.java

```
import java.util.HashMap;
import java.util.Map;

/**
 * A4 - Subarray Sum Equals K (Hostel Attendance-Change Log)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 * Prefix sums + hash map - O(n) time, O(n) space.
 *
 * A sliding window cannot be used here: with negative numbers the running sum is not
 * monotonic, so shrinking the window is no longer guaranteed to reduce the sum.
 */
public class SubarraySumEqualsK {

    /* sum(i+1..j) == prefix[j] - prefix[i], so count earlier prefixes equal to (running - k). */
    public static int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> prefixCounts = new HashMap<>();
        prefixCounts.put(0, 1); // the empty prefix: lets a subarray from index 0 count

        int runningSum = 0;
        int total = 0;

        for (int num : nums) {
            runningSum += num;
            total += prefixCounts.getOrDefault(runningSum - k, 0);
            prefixCounts.put(runningSum, prefixCounts.getOrDefault(runningSum, 0) + 1);
        }
        return total;
    }

    private static String format(int[] array) {
        StringBuilder text = new StringBuilder("[");
        for (int i = 0; i < array.length; i++) {
            text.append(array[i]);
            if (i < array.length - 1) text.append(", ");
        }
        return text.append("]").toString();
    }

    public static void main(String[] args) {
        int[][] inputs = { { 1, 1, 1 }, { 1, -1, 0 }, { 3, 4, 7, 2, -3, 1, 4, 2 } };
        int[] targets = { 2, 0, 7 };

        for (int t = 0; t < inputs.length; t++) {
            System.out.println("nums = " + format(inputs[t]) + ", k = " + targets[t]
                + " -> Count: " + subarraySum(inputs[t], targets[t]));
        }
    }
}
```

Output

```
nums = [1, 1, 1], k = 2 -> Count: 2  
nums = [1, -1, 0], k = 0 -> Count: 3  
nums = [3, 4, 7, 2, -3, 1, 4, 2], k = 7 -> Count: 4
```

Problem 5 — Find Minimum in Rotated Sorted Array

Problem Statement

A circular duty roster was originally sorted by join date, then rotated at some unknown point when the printed list started from a different staff member. Given only the resulting list, find the original earliest join date — without scanning every entry.

Approach

- The array is not fully sorted, so the middle element is compared against the rightmost element rather than against a target value.
- If `nums[middle] > nums[right]`, the rotation point must lie strictly to the right of middle, so left moves to middle + 1. Otherwise the minimum is at middle or to its left, so right moves to middle — never middle - 1, because middle itself could be the answer.
- The loop runs while `left < right`, so it exits exactly when the two pointers converge on the minimum. No equality case is needed since all elements are distinct.
- The un-rotated case falls out for free: for `[11, 13, 15, 17]` the middle is never greater than the right, so right keeps shrinking to index 0.
- middle is computed as `left + (right - left) / 2` rather than `(left + right) / 2`, which avoids integer overflow on very large bounds.
- $O(\log n)$ time and $O(1)$ space, as required — a linear scan would be correct but would not earn full marks.

Java Program — FindMinimumInRotatedSortedArray.java

```
/**
 * A5 - Find Minimum in Rotated Sorted Array (Circular Duty Roster)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 * Modified binary search -  $O(\log n)$  time.
 */
public class FindMinimumInRotatedSortedArray {

    /* Compares the middle element with the rightmost to decide which half holds the minimum. */
    public static int findMin(int[] nums) {
        int left = 0;
        int right = nums.length - 1;

        while (left < right) {
            int middle = left + (right - left) / 2;

            if (nums[middle] > nums[right]) {
                left = middle + 1; // the dip is strictly to the right of middle
            } else {
                right = middle; // middle could itself be the minimum, so keep it
            }
        }
        return nums[left]; // left and right have converged on the minimum
    }

    private static String format(int[] array) {
        StringBuilder text = new StringBuilder("[");
        for (int i = 0; i < array.length; i++) {
            text.append(array[i]);
            if (i < array.length - 1) text.append(", ");
        }
        return text.append("]").toString();
    }

    public static void main(String[] args) {
        int[][] inputs = { { 3, 4, 5, 1, 2 }, { 4, 5, 6, 7, 0, 1, 2 }, { 11, 13, 15, 17 }, { 2, 1 } };

        for (int[] nums : inputs) {
            System.out.println("nums = " + format(nums) + " -> Minimum: " + findMin(nums));
        }
    }
}
```


Output

```
nums = [3, 4, 5, 1, 2]  -> Minimum: 1  
nums = [4, 5, 6, 7, 0, 1, 2]  -> Minimum: 0  
nums = [11, 13, 15, 17]  -> Minimum: 11  
nums = [2, 1]  -> Minimum: 1
```